

AI-Driven User Behavior Analytics for Insider Threat Detection

Kelly Yue (ky387)

1. Introduction

In cybersecurity, the most dangerous risk does not come from external forces, but rather those hidden within. Insider threats are defined as security risks that originate from within an organization, where employees, contractors, or partners misuse authorized access to steal data, sabotage systems, or negligently expose information. Compared to external attacks, these threats are often harder to detect and significantly more costly. The average insider threat can cost organizations millions of dollars, including investigations, remediation, regulatory penalties, and lasting reputational harm. What makes the attacks so dangerous is not the sophistication, but rather the legitimacy. The attackers already have access. Usually, guardrails such as perimeter-based security tools, such as firewalls, intrusion detection systems, and access controls, are designed to keep outsiders out. However, to these threats, they are fundamentally blind.

Insider threats fall into three broad categories. Malicious insiders are individuals on the inside who intentionally steal, sabotage, or leak data for personal gain or revenge. Negligent insiders create risk through human error, such as falling for phishing, misconfiguring systems, or poor security practices. Compromised insiders are employees whose credentials are stolen by an external attacker or who have been manipulated or coerced into aiding an attack.

What makes all three types especially difficult to catch is the nature of these attacks. Most insider threats do not occur overnight, but rather develop slowly, over weeks, months, or even years. This gradual behavioral drift is nearly impossible to detect with static rule-based systems. Take, for example, an employee who begins accessing files outside their normal role, starts working at unusual hours, or ends an uptick of emails to a personal address may not trigger any individual alarm. It is the pattern over time that reveals the threat. Static rule-based detection systems cannot capture this. A rule that states, “flag anyone who logs in after midnight,” generates noise. It cannot differentiate between an employee working overtime on a deadline versus someone exfiltrating data before their resignation.

As a result, AI offers a fundamentally different approach. Rather than asking “is this a known bad,” AI asks “is this usual for this specific user.” An AI-driven system can learn an individual’s behavioral baseline, such as their typical login times, their usual email volume, their normal file access patterns, and flag behaviors that are abnormal. A shift from population-level rules to individual-level learning is what makes AI so useful to insider threat problems, as AI can identify subtle indicators such as unusual access times, spikes in data transfers, changes in communication patterns, and more before significant damage occurs.

This project addresses the insider threat detection problem by building an AI-driven User Behavior Analytics system trained on the CERT Insider Threat Dataset (r6.2) from Carnegie Mellon University. It applies a Long Short-Term Memory (LSTM) Autoencoder, which is suited for sequential, time-series data, to learn an individual user's normal behavioral patterns across five different dimensions. These are logon and logoff events, USB device usage, email activity, file transfers, and web browsing. Once trained, the model identifies days where a user's behavior deviates significantly from their own historical baseline, flagging these anomalies as potential insider threat indicators.

2. Background and Related Work

User and Entity Behavior Analytics (UEBA) is a cybersecurity solution that uses machine learning and algorithms to detect anomalies in the behavior of users, devices, and applications. It is achieved by building a baseline of “normal” behaviors within a user. Any suspicious activity that breaches this baseline is then used to identify insider threats.

The CERT Insider Threat Dataset is a synthetic and labeled benchmark dataset produced by Carnegie Mellon University CERT Division to simulate enterprise activity logs. It was created to provide a public and realistic baseline for research and development of behavioral anomaly detection methods. It models thousands of employees over months, providing benign background data and malicious scenarios. The dataset has evolved through several versions, with the latest release being v6.2, which is used for this project.

LSTMs are a type of recurrent neural network (RNN) used to solve the vanishing gradient problem, where gradients that help update weights become extremely small as they propagate backwards from the output layer to early layers. This causes early layers to stop learning, hindering the model's ability to train effectively. By using a cell state (long-term memory) and gating mechanisms (forget, input, output), LSTMs selectively retain or discard information over long time steps. It is due to the ability to retain information across a long sequence that LSTMs are being used for this project. IN the context of insider threat detection, this means the model can detect that a user who has been consistently logged in between 8 am and 6 pm for eight months is behaving anomalously when they suddenly begin accessing systems at 2 am. That 2 am login may not be suspicious by itself, but compared to the other login times, it being different from the baseline is what makes it suspicious. The temporal memory of the LSTM is what allows this detection to be possible.

Autoencoders are unsupervised neural networks that learn to compress data into a lower-dimensional latent representation and then reconstruct it to match the original input by minimizing reconstruction error. By training on normal behaviors to reconstruct its one input, at inference time, normal sequences are reconstructed accurately while anomalous sequences are

behaviors the model has never been trained to reconstruct, producing high reconstruction error. This error signal is the anomaly score.

3. Methodology

Raw event logs from the CERT r6.2 dataset were aggregated into daily feature vectors per user across all five activity streams. Rather than operating on individual events, the model learns from a summary of each user’s daily behavior. This includes: how many times the user has logged in, whether they accessed systems after hours, and how much data they transferred, which is then compressed into a fixed-length vector fed into a sequence model.

There are two different implementations of the preprocessing after the first version to get better results. In Version 2, the global normalization used in Version 1 was replaced with per-user z-score normalization. Ground truth labels were applied at the day level using the precise incident windows from answers/insiders.csv. A day was labeled as malicious if it fell within the documented start-to-end date range of that user’s malicious activity, rather than labeling all days of a malicious user’s activity as positive.

3.1 Feature Engineering

Version 1: Seven Features and Global Normalization

For the first version, for each user, for each day, it imputed one number per activity stream, which is a raw count of how much activity happened.

<i>Features</i>	<i>What it measures</i>
logon_count	Total login/logoff events
logon_after_hours	Logins before 8 am or after 6 pm
device_count	Total USB connect/disconnect events
email_count	Total emails sent
email_size_total	Total bytes of email traffic
file_count	Total file copies to removable media
http_count	Total web requests

After building the daily vectors, a single StandardScaler was fitted on the entire training population and applied to all users. When logon_count was normalized, it questioned: “How does this user’s count compare to everyone else’s average?” If a user normally logs in 20 times a day and logs in 25 times one day, that is a 25% increase for that specific user. However, compared to the global population average, it might be normal noise.

Version 2: Twelve Features and Per-User Normalization

The second version added five new features to Version One. These features were added by specific scenarios in the CERT r6.2 dataset, where malicious users exploit data late at night.

<i>Features</i>	<i>What it measures</i>	<i>Why is it new</i>
logon_count	Total login/logoff events	N/A
logon_after_hours	Logins before 8 am or after 6 pm	N/A
logon_hour_mean	Average hour of login	Captures shift in working hours
logon_hour_std	Std deviation of login hour	Captures consistency, which is low normally, and spikes when behavior changes
device_count	Total USB connect/disconnect events	N/A
device_after_hours	USB events outside business hours	Direct exfiltration signal
email_count	Total emails sent	N/A
email_size_total	Total bytes of email traffic	N/A
email_attachments	Total attachment count	Exfiltration via email
file_count	Total file copies to removable media	N/A
file_after_hours	File copies outside business hours	Direct exfiltration signal
http_count	Total web requests	N/A

In this version, normalization was done in a different way. Rather than scaling against the popular, this version scaled each user against themselves. This was done by grouping all the data by the "user" column and then calculating the average value for the specific feature, representing the number of files they download per day. Then the standard deviation is calculated for each user, which measures how much their behavior typically fluctuates. After this, it subtracts the user's average from their current activity and divides it by their typical fluctuation. What this means is that a value of +3.0 for device_count doesn't mean "this user connected their USB 3

times.” It means that “this user connected their USB 3 standard deviation above their own personal average.” A user who does 50 USB connections a day looks normal; a quiet user who suddenly does 50 in one night looks extremely irregular.

3.2 Train, Validation, and Test Split

The dataset was split at the user level. 70% of benign users were assigned to training, 15% to validation, and the remaining 15% of benign users, plus all five malicious users, were assigned to the test set. The training and validation sets contained only benign days, which ensured that the autoencoder learned exclusively from normal behavior. The test set contained all days for its assigned users, the malicious activity windows, which allowed for a realistic evaluation of anomaly detection performance.

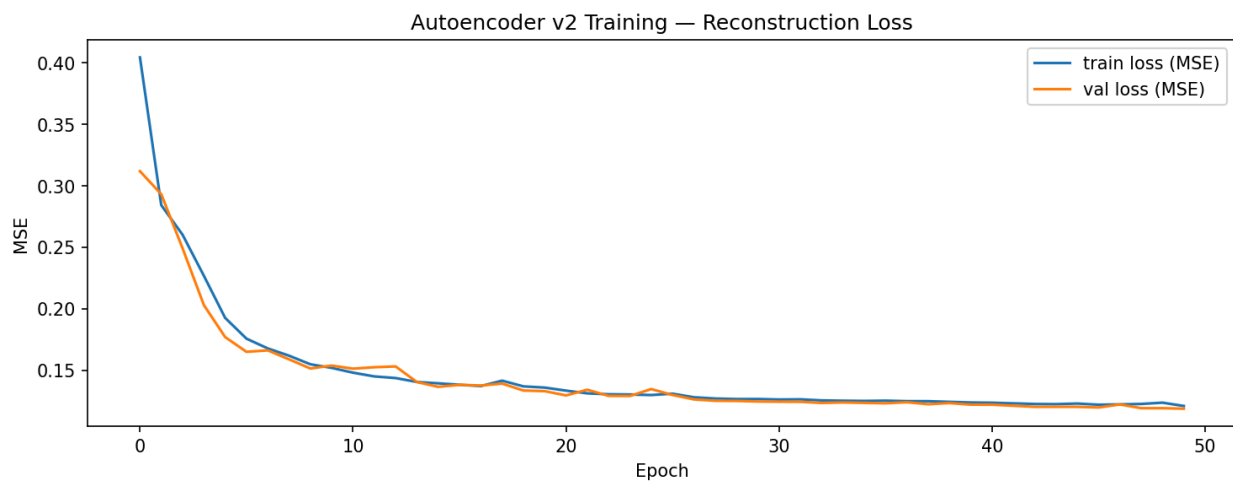


Figure 1 - Version 2 successfully learned a compact representation of normal user behaviors and converged without overfitting

3.3 Sequence Construction

Daily feature vectors were organized into a sliding window sequence of 30 consecutive days per user. This meant that each sequence represented a month of behavioral history and acts as a single input of the Long Short Term Memory Autoencoder. This 30-day window was picked to capture meaningful temporal context while remaining computationally tractable. The Malicious users were held out from the training and validation sets to prevent data leakage.

3.4 LSTM Autoencoder Architecture

As for the LSTM autoencoder architecture, both versions used a compact encoder-decoder architecture.

Version 1: The encoder consisted of two stacked LSTM layers with 64 units and 32 units, with a Dropout of 0.3 between them. This allowed compression for 30-day sequences into a

32-dimensional bottleneck vector. The decoder then repeated that vector across all 30 timesteps using a RepeatVector layer, which then reconstructed the original feature sequences through two LSTM layers, now with 32 and 64 units, followed by a TimeDistributed Dense output layer. This model was then trained with the Adam optimizer at a learning rate of 1×10^{-3} and mean squared error loss, with a batch size of 256 for up to 30 epochs.

Version 2: Used the same LSTM architecture as version one; however, widened it for more expressive representation learning. The encoder used three LSTM layers, 128, 64, and 16 units, with Batch Normalization and Dropout, which was 0.2 between each layer. The bottleneck was then narrowed from 32 to 16 dimensions. This allowed the encoder to compress behavioral sequences more aggressively. As a result, it makes it harder to reconstruct the out-of-distribution sequences. The decoder reflected the encoder with LSTM layers of 64 and 128 units, but with Batch Normalization. The learning rate was reduced to 5×10^{-4} for more careful convergence, and training was extended to 50 epochs with early stopping patience of 7 epochs.

Both versions used the same callbacks, where ModelCheckpoint saved the best validation loss, while EarlyStopping restored the best weights. ReduceLROnPlateau halved the learning rate when validation loss plateaued.

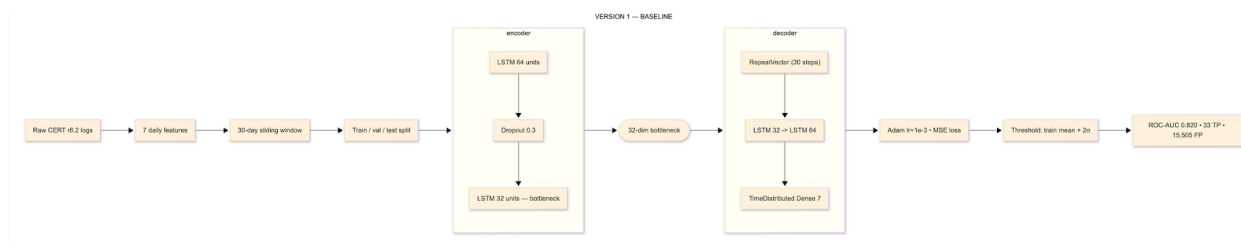


Figure 2 - Version 1 LSTM Autoencoder Architecture

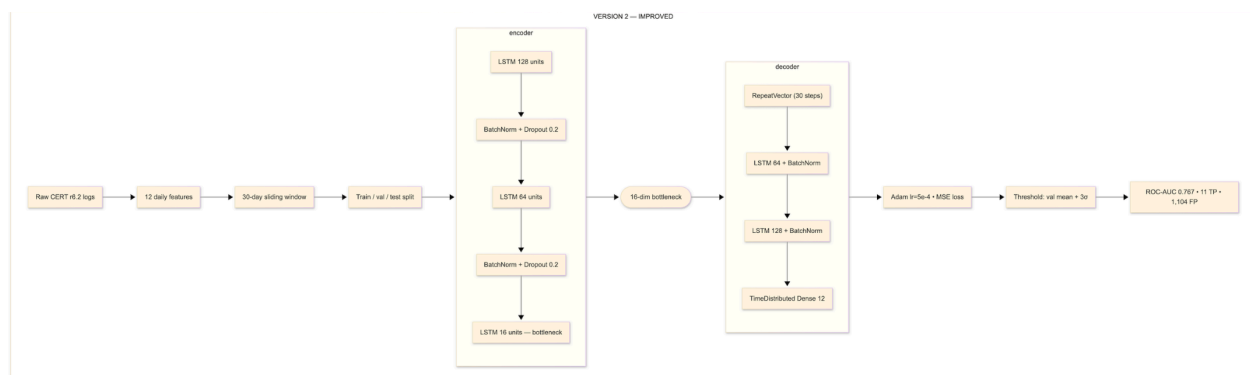


Figure 3 - Version 2 LSTM Autoencoder Architecture

3.5 Anomaly Detection Threshold

After training, a reconstruction error threshold was calculated to determine normal from anomalous sequences at inference time. In version 1, this threshold was computed from the training set reconstruction error mean as $+2\sigma$. In version 2, this was changed to use the validation set instead. This is mainly due to the validation set being benign-only and unseen during training. Due to this, it is a more reliable estimate of the model's reconstruction error floor on normal behavior. Additionally, the threshold was set at a mean of $+3\sigma$, which is more conservative and produces fewer false positives at the cost of some recall. At inference time, any sequences whose per-feature mean-squared reconstruction error exceeded the threshold are then flagged as a potential insider threat.

Results and Evaluation

Both models were trained successfully and converged without overfitting. Version 1 had reduced the reconstruction loss from about 0.43 on epoch 1 to about 0.07 by the final epoch. This means that there was a 77% reduction over 30 epochs. In version 2, there was a higher initial loss of around 0.30 due to the lower learning rates. It then converged to 0.07 as well, but this number required more epochs to reach this floor. Both cases depict that training and validation loss curves were tracked closely throughout, and there was no significant divergence, which would indicate that overfitting was present.

4.1 Detection Performance

<i>Metric</i>	<i>Version 1</i>	<i>Version 2</i>
Features	7, global normalization	12, per-user normalization
ROC-AUC	0.820	0.767
True Positives	33 / 99	11 / 99
False Positives	15,505	1,104
F1 Score (tuned threshold)	0.0042	0.0181
Anomaly Threshold	0.2178 (train mean + 2σ)	0.5080 (val mean + 3σ)

ROC-AUC (Receiver Operating Characteristics - Area Under the curve) measures how well the model ranks malicious sequences above benign ones across all possible thresholds. This means a score of 1.0 means that the model perfectly separates two classes. In Version 1, there is a ROC-AUC score of 0.820, which shows that the model has learned a signal, where it ranks malicious sequences higher than benign ones with a 82% reliability. The step-shaped ROC curve

reflects the small number of positive examples, which are 99 malicious sequences within 284,030 total. This is a known factor of this extreme class imbalance in this dataset.

True positives are the number of malicious sequences that the models correctly flagged. Of the 99 malicious sequences in the test set, Version 1 detected 33 and Version 2 detected 11. False positives are the number of benign sequences that the model incorrectly flagged as malicious. Version 1 showed 15,505 false alarms while Version 2 produced 1,104.

The FI Score is the mean of precision (measures what fraction of flagged sequences were actually malicious) and recall (measures what fraction of all malicious sequences were caught). This dataset is highly imbalanced, with malicious sequences accounting for less than 0.04% of all sequences.

Anomaly Threshold is the reconstruction error cutoff at which a sequence is flagged. The LSTM Autoencoder produces a continuous error score rather than a direct classification. This means that a threshold has to be picked to convert that score into a binary flag.

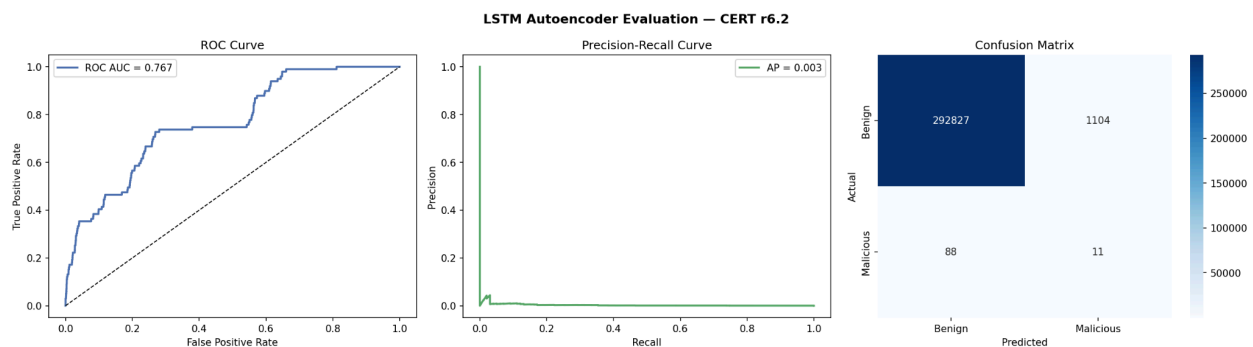


Figure 4 - ROC Curve, Precision-Recall Curve for Version 2

Discussion

This system learns behavioral representations from data without being hardcoded to look for any explicit rules. As a result, the LSTM brings temporal reasoning, where it is able to understand that the significance of an event relies on the history that preceded it. The autoencoder brings unsupervised anomaly detection, which is the ability to recognize deviations from learned normality without ever having seen a labeled attack.

5.1 Limitations

The training was conducted on a CPU rather than a GPU, resulting in training times exceeding eight hours per run. This had significantly limited the number of iterations and hyperparameter configurations that could be explored. The feature set remains simple.

This is because the daily aggregate counts do not fully capture the fine-grained behavioral signals that differentiate insider threat activity from normal high-variance behaviors. Take the example, when a user copies ten files to a USB drive at 2 am is far more suspicious than a user who copies ten files at 10 am. However, both produce the same `file_count` value of ten. By adding time-of-day distribution rather than after-hours binary flags, there would be a richer signal. Lastly, at the raw event level, malicious activity accounts for 0.14% of all 135 million events across the five activity streams. At the sequence level, only 99 of the 294,030 sequences are malicious, which represents 0.034% of the test set. This class imbalance places a ceiling on precision.

Conclusion

Overall, this project gave me experience in training large models in extremely large datasets. I would want to improve the accuracy score in the future. There were many challenges to this project, such as the extreme class imbalance limits precisions, and the feature sets it not optimized to have the best detection. The ROC-AUC does show that the LSTM Autoencoder has learned the structure of normal behaviors and therefore distinguishes them from anomalous sequences. This pipeline is similar to what commercial UEBA systems implement; therefore, it represents a meaningful step towards AI-driven insider threat detection that organizations could realistically deploy.

References:

Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. 2013 IEEE Security and Privacy Workshops. <https://doi.org/10.1109/SPW.2013.37>

Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. AAAI Workshops. <https://arxiv.org/abs/1710.00811>

Yuan, S., & Wu, X. (2021). Deep learning for insider threat detection: Review, challenges, and opportunities. Computers & Security, 104, 102221. <https://doi.org/10.1016/j.cose.2021.102221>

Le, D. C., Zincir-Heywood, N., & Heywood, M. I. (2020). Analyzing data granularity levels for insider threat detection using machine learning. IEEE Transactions on Network and Service Management, 17(1), 30–44. <https://doi.org/10.1109/TNSM.2020.2967721>

Le, D. C., & Zincir-Heywood, N. (2021). Anomaly detection for insider threats using unsupervised ensembles. IEEE Transactions on Network and Service Management, 18(2), 1152–1164. <https://doi.org/10.1109/TNSM.2021.3071928>

Lindauer, B. (2020). Insider Threat Test Dataset. Carnegie Mellon University. <https://doi.org/10.1184/R1/12841247.v1>